Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Maze Lab

Team Name: *Team #*	*Team Blockbuster* *T272*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Kumar, Tanmay – 2510370439* *2510370439@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Saxena, Sparsh – 2510370313* *2510370313@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Sethi, Hemang – 2510370341* *2510370341@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Maze Lab is a C++ application designed to combine core Data Structures and Object-Oriented principles into a unified software system. The software enables interactive maze navigation for a human player alongside automated graph-solving capabilities to compare human and algorithmic performance. Generation relies on Randomized DFS with an option for Kruskal's algorithm, while pathfinding is handled by BFS and A*. Performance metrics such as total steps, visited nodes, and timing are recorded on each run. We implemented the Strategy Pattern across generation and solving interfaces, allowing modular swap of underlying algorithms.

At roughly 65% completion, our core components—including grid construction, DFS generation, player handling, and BFS solving—are fully operational. Remaining tasks center on completing A*, Kruskal's algorithm, unified metrics, and comparative evaluation.

Updated Project Approach and Architecture

The environment uses a grid composed of Cell objects that store positional data, visit states, and wall boundaries. The primary Maze class maintains grid ownership alongside start and end nodes. Algorithms treat each cell as a graph vertex, treating open passages between adjacent cells as edges.

Modular design is enforced using abstract interfaces (`MazeGenerator` and `MazeSolver`) via the Strategy Pattern. While `DFSGenerator` and `BFSSolver` currently handle generation and pathfinding, upcoming modules (`KruskalGenerator` and `AStarSolver`) will plug directly into these existing base structures. Player control is isolated within a `Player` class that queries wall configurations prior to movement.

Each algorithm relies on distinct data structures: iterative DFS uses explicit stacks, BFS uses queues with parent pointer maps for path recovery, A* will leverage a priority queue guided by $f(n) = g(n) + h(n)$ with Manhattan distance, and Kruskal's algorithm will utilize a Disjoint Set Union (DSU) structure. Execution occurs within a single process in C++, rendering network protocols unnecessary. Console output handles current rendering, with graphics via SDL held as an optional phase.

Tasks Completed

Task Completed	Team Member
Cell and Maze classes (2D grid, wall logic)	Tanmay Kumar
Abstract MazeGenerator and MazeSolver interfaces	Tanmay Kumar
Iterative DFS generator using explicit stack	Sparsh Saxena
Player input handling and boundary collision logic	Hemang Sethi
Console visualization for maze state, player, and paths	Hemang Sethi
BFS pathfinding solver with parent tracking	Sparsh Saxena
End-to-end execution loop (generate → play → solve)	Tanmay Kumar

Challenges/Roadblocks

Maintaining Shared Wall Consistency: Updating adjacent cell walls individually caused synchronization mismatches where passages opened unilaterally. We resolved this by funneling all wall mutations through centralized `Maze` methods.

Preventing Stack Overflows: Recursive DFS generation risked exhausting the call stack on larger grid dimensions. We converted the generator to an explicit heap-allocated stack implementation.

Standardizing Evaluation Metrics: Fair comparison between BFS and A* requires uniform tracking. We designed a standardized result payload across the `MazeSolver` interface to collect identical timing and traversal statistics.

Cell-Based Wall Representation in Kruskal's Algorithm: Kruskal's algorithm operates on shared edges, whereas our representation stores walls internally per cell. We are addressing this by building an edge list strictly from right and bottom cell boundaries to prevent duplicate edge references during union operations.

Implementing A* and Kruskal requires auxiliary data structures (Priority Queue and DSU), so development on these modules has been prioritized ahead of final report write-ups.

Tasks Pending

Task Pending	Team Member (to complete the task)
A* pathfinder (Priority queue, Manhattan distance)	Sparsh Saxena
Disjoint Set Union (DSU) and Kruskal generator	Hemang Sethi
Metrics engine (path length, evaluated nodes, execution time)	Tanmay Kumar
Interactive menu and BFS vs A* execution side-by-side	Sparsh Saxena
Comprehensive test suite across all modules	All members
Final report preparation and project demonstration	All members
Optional: Graphical rendering using SDL2	Hemang Sethi (stretch goal)

Project Outcome/Deliverables

The primary deliverable is a C++ console application supporting interactive maze navigation and automated solving. It includes two generation methods (Randomized DFS, Kruskal's algorithm) and two pathfinders (BFS, A*), integrated via modular strategy interfaces. Output metrics include path distance, evaluated cells, and time taken. Deliverables also encompass full repository commit history, technical documentation, and a live demonstration. SDL-based graphics remain an optional stretch feature dependent on available time.

Progress Overview

Overall project completion stands at roughly 65%. Primary structures—grid modeling, DFS generation, collision checking, terminal display, and BFS traversal—are complete and functional. The remaining workload involves completing A*, Kruskal's generator, metric aggregation, algorithm comparisons, unit testing, and final documentation.

Development remains active without blocking issues, though Kruskal and A* require dedicated data structure setups before final benchmarking. Secondary graphics targets remain optional project goals.

Codebase Information

Repository: `https://github.com/Tanmay-git65/III-SEM-PBL`
Branch: master
Important commits:

- `b09bbff` Maze Lab PBL (initial commit)
- `e7b7868` Added C++ source code file
- `c9239f6` Added README.md file
- `9cea4fb` Added main file for Phase 2

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Maze connectivity (DFS)	Pass	Verifies all cells are reachable
Wall collision (Player)	Pass	Movement into walls properly rejected
BFS path validity	Pass	Path is continuous from start to goal
A* and Kruskal tests	Pending	Scheduled post-implementation
Benchmark & comparative evaluation	Pending	Scheduled after metrics implementation

Deliverables Progress

Interactive Console Application: In Progress (Core loop functional)
Randomized DFS Generator: Completed
Randomized Kruskal Generator: In Progress
BFS Pathfinder: Completed
A* Pathfinder: In Progress
Metrics & Comparison Engine: Pending
Test Suite & Final Report: Pending
SDL2 Graphical Engine (Optional): Pending